

ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No. 34 of year 2010
Recognised by the University Grants Commission (UGC), New Delhi

DATABASE DESIGN ANALYSIS

BACHELORS OF COMPUTER APPLICATION- III

NAME: PREKSHA V

REGISTER NUMBER: 2511021240063

SUBMITTING TO,

RENITA JOY R

1. Introduction

TheCab BookingManagement System designed in Assignments 1 and 2 contains eight core tables: CUSTOMER, DRIVER, CAB, BOOKING, TRIP, PAYMENT, RATING, and DRIVER_LOCATION. These tables represent customer details, driver details, vehicles, reservations, journeys, payments, feedback, and driver location updates.

The purpose of this analysis is to check whether the current design stores facts in the correct places, identify functional dependencies, explain possible insertion, deletion, and update anomalies, and recommend improvements. The analysis is based on the schema and relationship assumptions documented inthe earlierassignments.

2. Existing Database Structure

Table	Primary key	Importantattributes / foreign keys
CUSTOMER	Customer_ID	Name, Phone, Email, Address
DRIVER	Driver_ID	Name, Phone, License_Number, Status
CAB	Cab_ID	Driver_ID (FK), Vehicle_Number, Cab_Type, Model, Status
BOOKING	Booking_ID	Customer_ID (FK), Driver_ID (FK), Cab_ID (FK), Booking_Date, Pickup_Location, Drop_Location, Booking_Status
TRIP	Trip_ID	Booking_ID (FK), Start_Time, End_Time, Distance, Fare, Trip_Status
PAYMENT	Payment_ID	Booking_ID (FK), Amount, Payment_Method, Payment_Date, Payment_Status
RATING	Rating_ID	Booking_ID (FK), Customer_ID (FK), Driver_ID (FK), Rating, Review
DRIVER_LOCATION	Location_ID	Driver_ID (FK), Latitude, Longitude, Updated_Time

Design assumptions: Assignment 2 specifies Driver–Cab as 1:1, Booking–Trip as 1:1, Booking–Payment as 1:1, and Booking–Rating as 1:0..1. These assumptions are used below. In a real booking platform, some of these relationshipsmay needto be expanded.

3. Data Redundancy and Potential Anomalies

Dataredundancy occurs whenthesamefact is storedin morethan oneplace.Redundancy is not automatically an error, but duplicated facts can become inconsistent when records are inserted, updated, or deleted.

3.1 Potential Redundancy in the Current Design

Area	Why it may be redundant	Potential risk
BOOKING.Driver_ID and CAB.Driver_ID	The cab table also records an associated driver. If the design treats CAB.Driver_ID as the permanent driver for that cab, the driver may be represented in both tables.	The booking could name a driver different from the driver stored against the cab. However, keeping the driver assigned at booking time can be valid for historical records if assignments change over time.
RATING.Customer_ID and RATING.Driver_ID	Both IDs may be obtained by following RATING.Booking_ID to BOOKING, because BOOKING already stores Customer_ID and Driver_ID.	A rating may point to a booking but list a different customer or driver, producing contradictory records.
Repeated customer or driver details in transactions	If customer names, phone numbers, or driver details were copied into BOOKING, TRIP, PAYMENT, or RATING, the same facts would be stored repeatedly.	Changing a phone number or name would require updates in many records. The current schema mostly avoids this by keeping profile details in separate tables.

3.2 Insertion Anomaly

Aninsertion anomalyoccurs when a fact cannot be added without adding an unrelated fact, or when adding a record requires unnecessary duplicate data.

Example: If a driver can be registered only when a cab is assigned, the system may be unable to store a newly hired driver before a vehicle is assigned. Similarly, if driver details were stored only in BOOKING, a driver could not be recorded until the driver receives a booking.

In the current design, the separate DRIVER and CAB tables reduce this problem. The application should allow a DRIVER record to be inserted independently and should not require a BOOKING record to register a customer or driver.

3.3 Deletion Anomaly

A deletionanomalyoccurswhen removing one record unintentionally removes another fact that should be retained.

Example: If driver details were stored only in BOOKING, deleting the driver's last booking could also remove the only stored copy of that driver's name and phone number. Likewise, if a customer's profile were stored only alongside bookings, deleting the customer's last booking could erase the profile. Keeping CUSTOMER and DRIVER as separate tables helps prevent this. Deleting a booking should not automatically delete the associated customer, driver, or cab master record. Foreign-key deletion rules should be chosen carefully.

3.4 Update Anomaly

An updateanomalyoccurswhen the same fact appears in multiple rows or tables and only some copies are changed.

Example: if a driver's phone number is copied into several booking records, updating only one row leaves conflicting phone numbers. In this schema, a more specific risk is the driver assignment: CAB.Driver_ID and BOOKING.Driver_ID could disagree if the relationship is treated as a current fixed assignment rather than an assignment recorded for a particular booking.

A rating may also become inconsistent if RATING.Customer_ID or RATING.Driver_ID does not match the customer and driver associated with its Booking_ID.

4. Functional Dependencies

Afunctional dependencyX →Y means thatthevalue of attribute (or attribute set) X determines exactly one value of Y in the database. Primary keys determine the other attributes in their rows. The dependencies below follow the intended meaning of the Assignment 2 schema; candidate-key assumptions are noted where needed.

Table	Main functional dependency	Explanation / condition
CUSTOMER	Customer_ID → Name, Phone, Email, Address	A customer ID identifies one customer profile.
DRIVER	Driver_ID → Name, Phone, License_Number, Status	A driver ID identifies one driver profile.
CAB	Cab_ID → Driver_ID, Vehicle_Number, Cab_Type, Model, Status	A cab ID identifies one vehicle record. Driver_ID is determined by Cab_ID under the current schema.
BOOKING	Booking_ID → Customer_ID, Driver_ID, Cab_ID, Booking_Date, Pickup_Location, Drop_Location, Booking_Status	A booking ID identifies one reservation and its stored assignment and status.
TRIP	Trip_ID → Booking_ID, Start_Time, End_Time, Distance, Fare, Trip_Status	A trip ID identifies one trip. If Booking–Trip is truly 1:1, Booking_ID should also be UNIQUE and determine the trip attributes.
PAYMENT	Payment_ID → Booking_ID, Amount, Payment_Method, Payment_Date, Payment_Status	A payment ID identifies one payment. If Booking–Payment is truly 1:1, Booking_ID should also be UNIQUE.
RATING	Rating_ID → Booking_ID, Customer_ID, Driver_ID, Rating, Review	A rating ID identifies one feedback record. Under the one-rating-per-booking rule, Booking_ID should be UNIQUE.
DRIVER_LOCATION	Location_ID → Driver_ID, Latitude, Longitude, Updated_Time	A location ID identifies one recorded location update. A driver can have many location records.

4.1 Additional Candidate Dependencies

If the business rules require each phone number, email address, driving licence number, and vehicle registration number to be unique, the following additional dependencies may be enforced: Phone → Customer_ID (or Driver_ID in DRIVER), Email → Customer_ID, License_Number → Driver_ID, and Vehicle_Number → Cab_ID. These should be treated as candidate-key rules only if the application confirms uniqueness; they are not guaranteed by the assignment description alone.

4.2 Dependencies Across Tables

Because BOOKING stores Customer_ID and Driver_ID, Booking_ID determines the customer and driver assigned to that booking. Therefore, if RATING.Booking_ID identifies a unique booking, the customer and driver for the rating can be checked through BOOKING. Storing those IDs again in RATING introduces a consistency rule that the database must enforce.

5. How Poor Design Causes Anomalies

Repeated customer, driver, or cab details create redundancy and make updates difficult.

Poor design choice	
Storing master data with every transaction	
Putting multiple facts into one table	Unrelated facts become dependent on one another, so inserting or deleting a transaction can affect profile or vehicle information.
Duplicating attributes that can be derived through a foreign key	Two copies may disagree, such as the driver ID in a rating and the driver assigned to its booking.
Not enforcing UNIQUE constraints for intended 1:1 relationships	Multiple trip or payment rows may be created for one booking even though the design says one-to-one.
Weak foreign-key and deletion rules	Orphan records or accidental deletion of useful records may occur.

The main principle is that each fact should have one authoritative place to be stored. Related tables should be connected using primary and foreign keys, and business rules should be enforced with constraints rather than relying only on users or application code.

6. Proposed Improvements to the Database Design

Theexistingdesign already separatesthemajorentities into different tables,whichisagood starting point. The following changes would improve consistency and make the intended relationships explicit.

Improvement	Recommended action	Benefit
1. Enforce primary and foreign keys	Keep the existing primary keys and foreign-key references. Ensure all foreign keys reference existing parent rows.	Prevents duplicate identifiers and orphan records.
2. Enforce one-to-one relationships	Add UNIQUE constraints to TRIP.Booking_ID and PAYMENT.Booking_ID if each booking must have at most one trip and one payment. Add UNIQUE to RATING.Booking_ID if each booking may receive at most one rating.	Makes the database enforce the cardinalities described in Assignment 2.
3. Review RATING attributes	Consider removing RATING.Customer_ID and RATING.Driver_ID and obtaining them through RATING.Booking_ID → BOOKING, provided a rating always concerns that booking's customer and driver. If retained, add validation to ensure both IDs match BOOKING.	Avoids duplicated relationship data and inconsistent ratings.
4. Clarify driver–cab assignment	If drivers and cabs can be reassigned over time, do not rely on CAB.Driver_ID as the historical driver for every booking. Keep BOOKING.Driver_ID as the driver assigned for that specific booking, and model current or dated driver–cab assignments separately if required.	Preserves booking history and avoids ambiguity when assignments change.
5. Add candidate-key constraints	Where the business rules require uniqueness, set UNIQUE constraints for CUSTOMER.Phone/Email, DRIVER.License_Number, and CAB.Vehicle_Number. Allow NULL only where the application permits it.	Prevents duplicate contact, licence, or vehicle identifiers.
6. Validate data domains	Use suitable data types and CHECK constraints: Rating between 1 and 5; Distance and Fare non-negative; End_Time not earlier than Start_Time; and status/method values restricted to permitted options.	Prevents invalid or contradictory values.
7. Define deletion and update rules	Use RESTRICT/NO ACTION for important master records where history must remain. Use CASCADE only when dependent rows should genuinely be deleted together; use SET NULL only where the relationship is optional.	Reduces accidental loss of important transaction history.
8. Normalize to at least 3NF	Check that every non-key attribute depends on the key, the whole key, and nothing but the key. Keep customer, driver, and cab descriptive details in their own tables rather than copying them into transactions.	Reduces redundancy and insertion, deletion, and update anomalies.

6.1 Suggested Refined Schema

A practical refined schema, preserving the main structure of Assignments 1 and 2, is shown below. The schema keeps the driver assigned at booking time and avoids duplicating customer/driver IDs in RATING when those can be derived from the booking.

Table	Suggested attributes and constraints
CUSTOMER	Customer_ID (PK), Name, Phone, Email, Address
DRIVER	Driver_ID (PK), Name, Phone, License_Number, Status
CAB	Cab_ID (PK), Vehicle_Number (UNIQUE), Cab_Type, Model, Status; current Driver_ID only if the current 1:1 assignment rule is retained
BOOKING	Booking_ID (PK), Customer_ID (FK), Driver_ID (FK), Cab_ID (FK), Booking_Date, Pickup_Location, Drop_Location, Booking_Status
TRIP	Trip_ID (PK), Booking_ID (FK, UNIQUE), Start_Time, End_Time, Distance, Fare, Trip_Status

Table	Suggested attributes and constraints
PAYMENT	Payment_ID (PK), Booking_ID (FK, UNIQUE), Amount, Payment_Method, Payment_Date, Payment_Status
RATING	Rating_ID (PK), Booking_ID (FK, UNIQUE), Rating, Review
DRIVER_LOCATION	Location_ID (PK), Driver_ID (FK), Latitude, Longitude, Updated_Time

Important: The UNIQUE constraint on Booking_ID in TRIP, PAYMENT, and RATING matches the assignment's one-to-one or one-to-optional-one assumptions. If the system later supports multiple payment attempts, split payments, or multiple ratings per booking, those constraints and relationships should be revised to match the new requirements.

7. Normalization Analysis

Thedatabase is broadly organized aroundseparate entities, which supports normalization. The recommended changes strengthen the design as follows:

First Normal Form (1NF): Each field should contain a single value, such as one phone number or one location coordinate per field. Repeated location updates are stored as separate DRIVER_LOCATION rows rather than multiple coordinates in one cell.

Second Normal Form (2NF): Each table uses a single-attribute primary key, so partial dependency on part of a composite primary key is not expected in the listed schema. If composite keys are introduced later, all non-key attributes must depend on the complete key.

Third Normal Form (3NF): Non-key attributes should depend on the key and not on other non-key attributes. Keeping customer, driver, and cab details in their own tables helps satisfy this principle. Removing or validating duplicated customer/driver IDs in RATING further improves consistency. Normalization should preserve the business meaning of the data. In particular, BOOKING.Driver_ID records the driver assigned to a booking, while CAB.Driver_ID may represent a current vehicle assignment; those are not necessarily the same fact if assignments can change over time.

8. Conclusion

The Cab Booking Management System has a reasonable relational foundation because customer, driver, cab, booking, trip, payment, rating, and location data are stored in separate tables. This separation already reduces many common forms of redundancy.

The main risks identified are the possible duplication of driver assignment data between CAB and BOOKING, duplicated customer and driver references in RATING, and the need to enforce the one-to-one relationships stated in the design. If these issues are not handled, the database may contain inconsistent assignments, ratings linked to the wrong participants, or multiple records where only one is intended.

The recommended improvements are to define functional dependencies clearly, enforce primary and foreign keys and appropriate UNIQUE/CHECK constraints, review redundant rating attributes, clarify whether driver–cab assignments are current or historical, and maintain normalization through 3NF.

These changesmakethedatabase moreaccurate,reliable, and easier to maintain.

9. Summary of Key Findings

Requirement	Finding
Data redundancy	Potential overlap between CAB.Driver_ID and BOOKING.Driver_ID; RATING repeats customer and driver IDs already associated with BOOKING.
Insertion anomaly	Can arise if master records are stored only with transactions; separate CUSTOMER and DRIVER tables reduce this risk.
Deletion anomaly	Can arise if deleting a transaction also removes the only copy of a customer or driver profile.
Update anomaly	Can arise when duplicated facts are changed in only one location, producing inconsistent records.
Functional dependencies	Each primary key determines the non-key attributes of its table; additional candidate dependencies depend on confirmed uniqueness rules.
Improvements	Use constraints, avoid unnecessary duplicated attributes, clarify assignment history, validate data, and maintain 3NF.

Reference basis

Thisanalysis isbasedonthe Cab Booking Management System schema, entity attributes, keys, and cardinalities documented in the user's DBMS Assignment 1 and Assignment 2. No external dataset or implementation was assumed.